# Classifiers: Predicting Who Will Do What

MKT 566 · Fall 2026

Davide Proserpio
USC Marshall School of Business

# Before We Start

# Housekeeping

- **Project proposals**: feedback and grades are posted on Brightspace (Assignments → Project: Proposal presentation)
- **Homework 3** due **Thursday, Oct. 29**
- **Thursday**: guest speaker **Hong Zhang**, Marketing Data Science Manager at **Netflix**. Come with questions
- Today: **classifiers**, models that predict who will do what

# From explaining to predicting

So far every model answered "**what drives** Y": what drives price, what drives the chance a listing is a gem. We read **coefficients** and told a story about the market.

Today we ask a different question: "**who will do it?**"

- Which customers will **cancel** next month? (churn: yes/no)
- Who will **click** this ad? Who will **use** the coupon?
- Which reviews are **fake** and which are organic?

> Same tools, different goal: we no longer care only about *why*. We care whether the model can **call the right names**, customer by customer, so marketing can act: who gets the retention offer, the ad, the audit.

# What is a classifier?

A **classifier** is a model that assigns each customer (each row) to a **class**, a bucket:

| Kind | Classes | Marketing example |
|---|---|---|
| **Binary** | 2 | churn vs. stay, click vs. no click |
| **Multi-class** | 3 or more | which ad (A, B, or C), which product category |
| **Ordinal** | ordered | star rating (1 to 5) |

- Inputs are **features**: tenure, logins, past purchases, ad impressions
- Output is a **predicted class**, usually through a **probability**: the model says $p(\text{churn}) = 0.72$, and a **decision rule** turns it into a label: if $p \geq 0.5$, predict "churn"

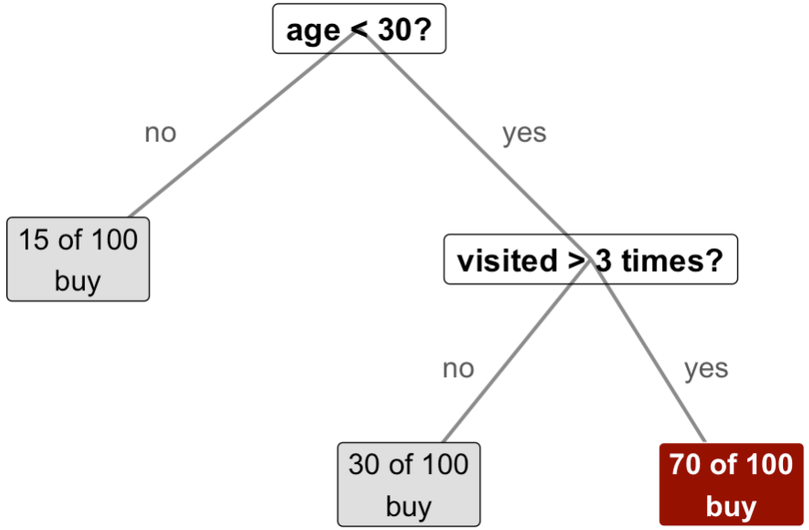
> You already built one. The **logit** from week 5 outputs a probability: add a cutoff, and it is a classifier. Today: how to **judge** it, and what else is in the toolbox.

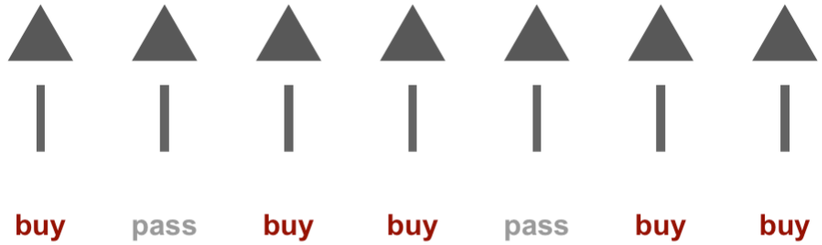
# The toolbox, in plain words

| Model | How it decides | Can you read it? |
|---|---|---|
| **Logistic regression** | one score from a weighted sum of features, turned into a probability | yes: coefficients and odds ratios (week 5). The usual **baseline**: the simple model you start from and try to beat |
| **Decision tree** | a flowchart of simple rules ("age < 30 and visits > 3: likely buyer") | yes: show the tree to a manager |
| **Random forest** | hundreds of trees, each on a random slice of the data; they **vote** | partly: it ranks which features matter, but there are no rules to read |
| **Boosting** (XGBoost) | trees built one after another, each fixing the previous one's mistakes | partly: many settings to get right |
| **Neural network** | layers that learn combinations of features | no: a black box |

The tradeoff that runs through the whole table: **better predictions vs. a story you can read**. Forests and boosting usually predict a little better; the logit is the easiest to explain. Start with the logit, then check whether the fancy model beats it by enough to matter.

# Trees and forests, drawn




**5 of 7 trees say "buy"**
**the forest predicts buy: 5/7 = 71%**

- A tree sorts customers by asking **one question at a time**; every customer lands in exactly **one leaf** (one of the boxes at the end)
- The leaf's history is the prediction: of the 100 past customers in the right-most leaf, **70 bought**, so a new customer there gets **p = 0.70**
- A **random forest** grows many trees, each on a **random slice** of the data, so the trees disagree a little
- For a new customer **every tree votes**, and the forest averages the votes: one tree memorizes quirks, the **average washes them out**

An illustration with made-up numbers: the splits, the counts, and the votes are for reading practice, not from data.

# Forests and boosting: one table

Both combine many trees. They differ in **how** the trees are built:

| | Random forest | Boosting (XGBoost) |
|---|---|---|
| How it works | many trees **in parallel**, each on a random sample; predictions are **averaged** | trees **one after another**; each new tree fixes the previous mistakes |
| What it improves | steadier predictions: one odd tree cannot sway the average | sharper predictions: each tree fixes what the last one missed |
| Tuning | few knobs, works well out of the box | many knobs: how fast it learns, how deep each tree goes, how many trees |
| Risk of memorizing quirks | lower | higher, if not tuned carefully |
| When to use | a fast, strong baseline on noisy data | chasing the best predictions, with time to tune |

# Scoring a Classifier

# Every prediction lands in one of four boxes

Suppose we predict whether each customer **churns (yes/no)**, and then watch what actually happens. Four things are possible:

| | Predicted: churn | Predicted: stay |
|---|---|---|
| **Actually churned** | **true positive (TP)** | **false negative (FN)**, we missed one |
| **Actually stayed** | **false positive (FP)**, false alarm | **true negative (TN)** |

This 2-by-2 count is the **confusion matrix**: every customer falls in exactly one cell.

> The two mistakes are **not the same mistake**. A false positive wastes a retention offer on someone who was staying anyway. A false negative lets a churner walk out the door. Which one costs more depends on the business.

# A worked example: 200 customers

Our classifier made a call for 200 customers, and then we watched what happened:

| | Predicted: churn | Predicted: stay |
|---|---|---|
| **Actually churned** | 50 (TP) | 10 (FN) |
| **Actually stayed** | 20 (FP) | 120 (TN) |

Three questions, three metrics. Compute them from the table:

- **Accuracy**: what share of all 200 calls were correct?
- **Precision**: of the customers we **called churners**, how many were?
- **Recall**: of the customers who **actually churned**, how many did we catch?

Take a minute: each answer is one fraction, read straight off the table.

# The same example, computed

| | Predicted: churn | Predicted: stay |
|---|---|---|
| **Actually churned** | 50 (TP) | 10 (FN) |
| **Actually stayed** | 20 (FP) | 120 (TN) |

- **Accuracy** $= \frac{TP+TN}{200} = \frac{50+120}{200} = 0.85$: the model gets 85% of all calls right
- **Precision** $= \frac{TP}{TP+FP} = \frac{50}{50+20} = 0.71$: when it says "churner", it is right 71% of the time. The other **29% of retention offers are wasted** on customers who were staying anyway
- **Recall** $= \frac{TP}{TP+FN} = \frac{50}{50+10} = 0.83$: it catches 83% of the real churners. The other **17% slip away unflagged**

# When accuracy lies

Churn is usually **rare**. Say 10,000 customers, **5% churn**: 500 churners, 9,500 stayers. Compare two models:

| Model | Predicted churn | TP | FP | FN | TN | Accuracy | Precision | Recall |
|---|---|---|---|---|---|---|---|---|
| "Nobody churns" | 0 | 0 | 0 | 500 | 9,500 | **95%** | undefined | **0%** |
| A useful model | 600 | 300 | 300 | 200 | 9,200 | **95%** | 50% | 60% |

- The do-nothing model scores **95% accuracy** by predicting "stay" for everyone, and catches **zero** churners. Accuracy 95% here means nothing
- The useful model has the **same accuracy**, but it finds 300 of the 500 churners. Precision and recall see the difference; accuracy cannot

Why "undefined": the first model never says "churn", so there are no churn calls to grade. Precision would be 0 divided by 0.

> When the "yes" class is rare (churners, clickers, fraud), **never judge a model by accuracy alone**. Look at precision and recall.

# One number, when you need one: F1

**F1** combines precision and recall into one number between 0 and 1:

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- For the useful model above: $F1 = 2 \cdot \frac{0.50 \times 0.60}{0.50 + 0.60} = 0.55$
- For the do-nothing model: recall is 0, so **F1 = 0**. The 95% accuracy disappears
- F1 is high only when **both** precision and recall are reasonably high: few wasted offers **and** few missed churners

Use F1 to rank models with one number when one class is rare. But for decisions, keep precision and recall separate: they price two different mistakes.

# Will It Work on New Customers?

# The trap: memorizing the past

The goal is to predict **new, unseen customers**. The danger is a model that **memorizes** the old ones instead:

- A flexible model can learn quirks of the data it learned from, like a tree that learns “customer #123 always buys”. Perfect on the past, **useless** for the future
- This is **overfitting**: accuracy on that same old data looks great (even 100%), and predictions on new customers fail

> A model is only as good as its performance on data it has **never seen**. Grading it on the data it learned from is letting it grade its own homework.

# The fix: hold some data out

**Split the data before fitting** (usually 70/30 or 80/20):

| Set | What it is for |
|---|---|
| **Training set** | fit the model: estimate coefficients, grow the trees |
| **Test set** | held out, never touched during fitting: it plays the role of **new customers** |

Then compare:

- Accuracy on the training set **much higher** than on the test set: **overfitting**
- The two similar: the model **generalizes**, it works beyond the data it learned from

> A churn example with real calendar time: train on 2019 to 2024 customers, test on 2025 customers. If it holds up on 2025, we can trust it for 2026.

# Cross-validation: a fairer check

One split can get lucky. **K-fold cross-validation** repeats the check:

1. Split the training data into **k** equal parts, called folds (say k = 5)
2. For each fold: fit the model on the other 4, check the metric (say, precision) on the held-out fold
3. **Average** the metric across the 5 folds
4. Pick the model (or tuning) with the best average, then refit on all the training data

- Every row gets used for training **and** for validation (in turns)
- Prevents overfitting to **one lucky split**, and compares models fairly

We will not run cross-validation by hand today: R does it in a few lines, and the exercise sticks to one train/test split. What matters is the idea: **never grade a model on the data it trained on.**

# Very rare classes: rebalancing the training data

Rare classes break accuracy: a few slides ago, the do-nothing model scored **95%** and caught **zero** churners. Our first fix was better grading: judge the model by **precision** (how good its "churn" calls are) and **recall** (how many churners it catches), not by accuracy. When the "yes" class is **truly scarce** (1% fraud, not 18% churn), there is a second lever: change what the model **trains on**.

| Option | What it does |
|---|---|
| **Oversample** the rare class | repeat the existing churners (or create lookalike ones) in the training set until the model sees them often |
| **Undersample** the common class | drop stayers from the training set |
| **Class weights** | keep the data as it is; tell the model that missing a churner is a bigger mistake than flagging a customer who was going to stay anyway |

- All three push the model to **say "churn" more often**, instead of playing it safe with "stay" (the answer that is almost always right when churn is rare). So it **catches more real churners (recall up)** and **wastes more calls on stayers (precision down)**. Nothing is free; you are choosing which mistake to make more often

# Rebalancing: the two rules

- Rule 1: rebalance the **training set only**. The test set's job is to show how the model will do in the real world, so it must stay as imbalanced as reality
- Rule 2: after rebalancing, the **scores come out too high**. Why: a model's score for a customer is, roughly, the churn share among the **training customers who look like them**, and rebalancing faked that share
  - Say 1 of 10 lookalikes churns: the true chance is **10%**. Oversample by duplicating that one churner until he appears 9 times: the training data now shows 9 churner rows against the 9 stayers, so the model counts **9 of 18 = 50%**. The world did not change; the photocopies did that
- The inflation hits everyone, so the **order is still right**: riskier customers still score higher. Use the scores to decide **who to call first**; just do not read 0.50 as "a 50% chance"

> Rebalancing does not create information that is not in the data. Try the honest version first: keep the data as it is, grade with precision and recall, and if the model flags too few churners, lower the bar in the decision rule (predict churn at a score of 0.2 instead of 0.5).

# A Churn Model, End to End

# Our running example: churn at a subscription business

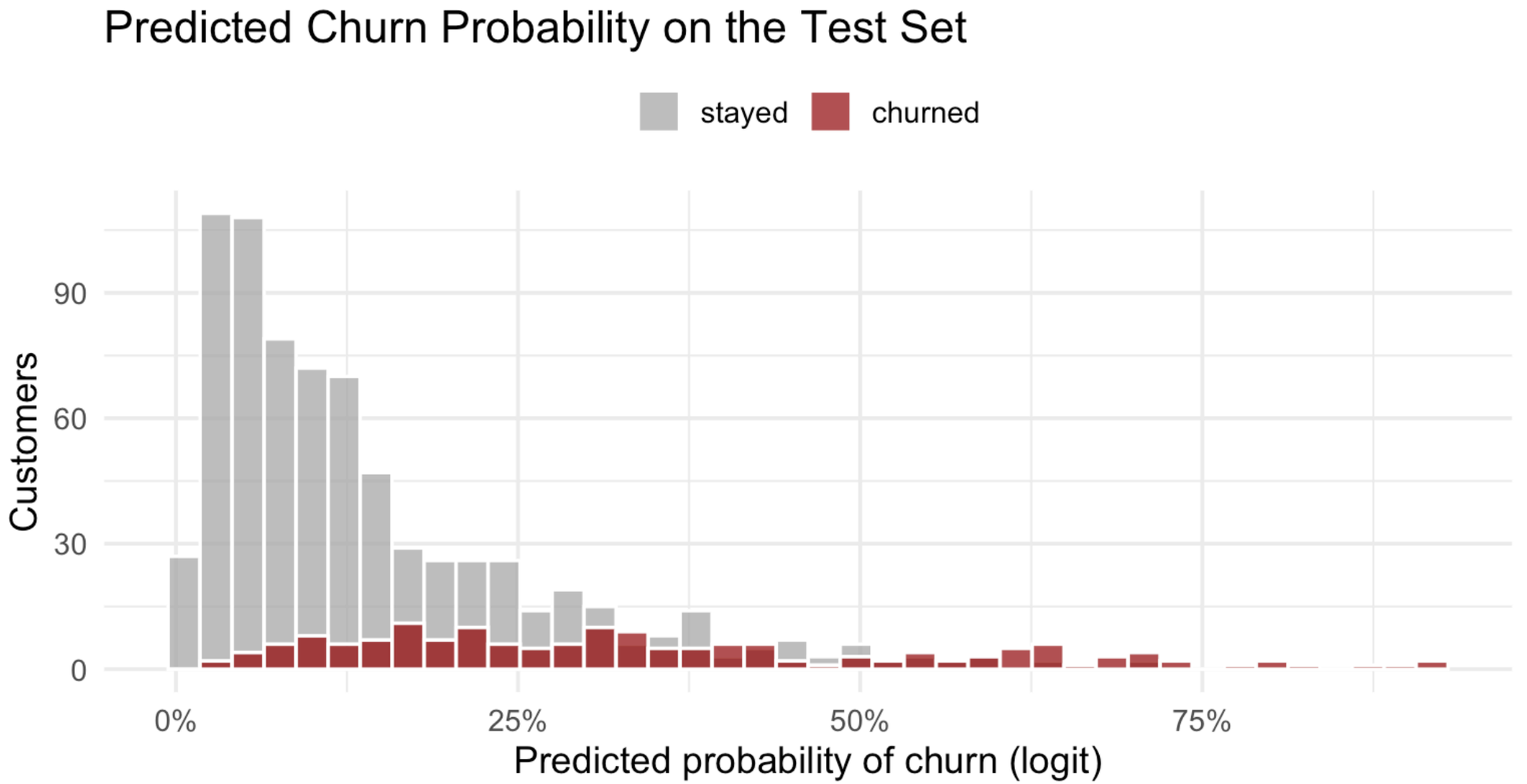
**Laptops out: from here we build the model together in R.** Download `w9-code.zip` from the course page, open `w9-1-classifiers-class.R`, and run along; every chart and table on the slides comes out of that script.

The data: 3,000 customers of a subscription service, **18.3% churned**. For each customer we see:

| Feature | Why it should matter |
|---|---|
| tenure (months) | long-time customers are safer |
| logins, last 30 days | fewer logins, higher risk |
| support tickets, last 30 days | more tickets, higher risk |
| email engagement (0 to 1) | engaged customers are safer |
| price increase, last 60 days | a price hike raises risk |
| plan downgrade | a downgrade is a warning sign |
| region | small differences across markets |

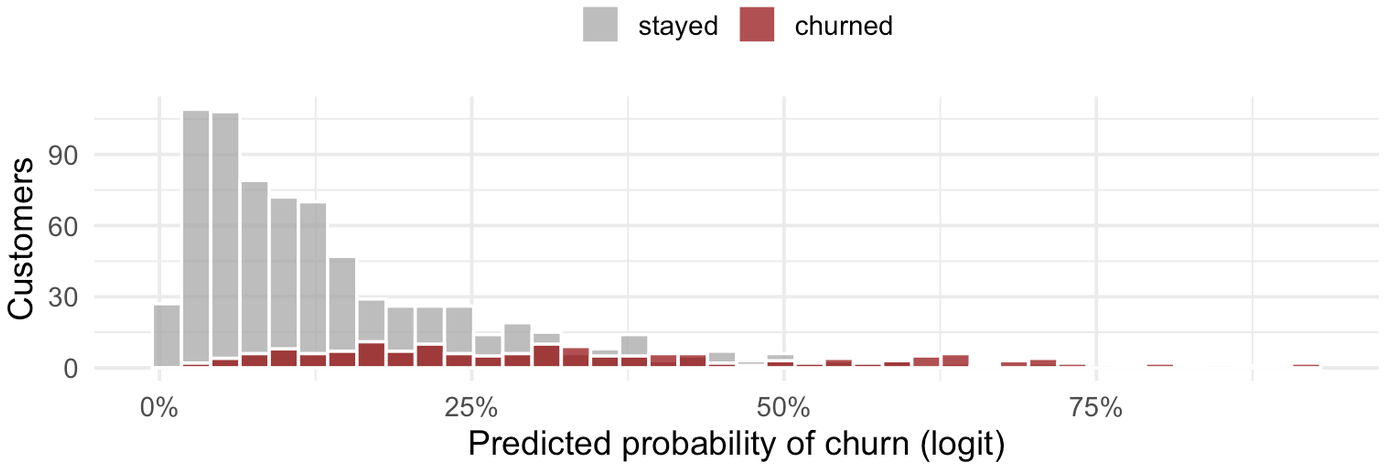
Setup: **70/30 split** (2,099 train, 901 test), then a **logit** on the training set. Every number that follows is computed on the **test set**: customers the model has never seen.

# The model's output is a score



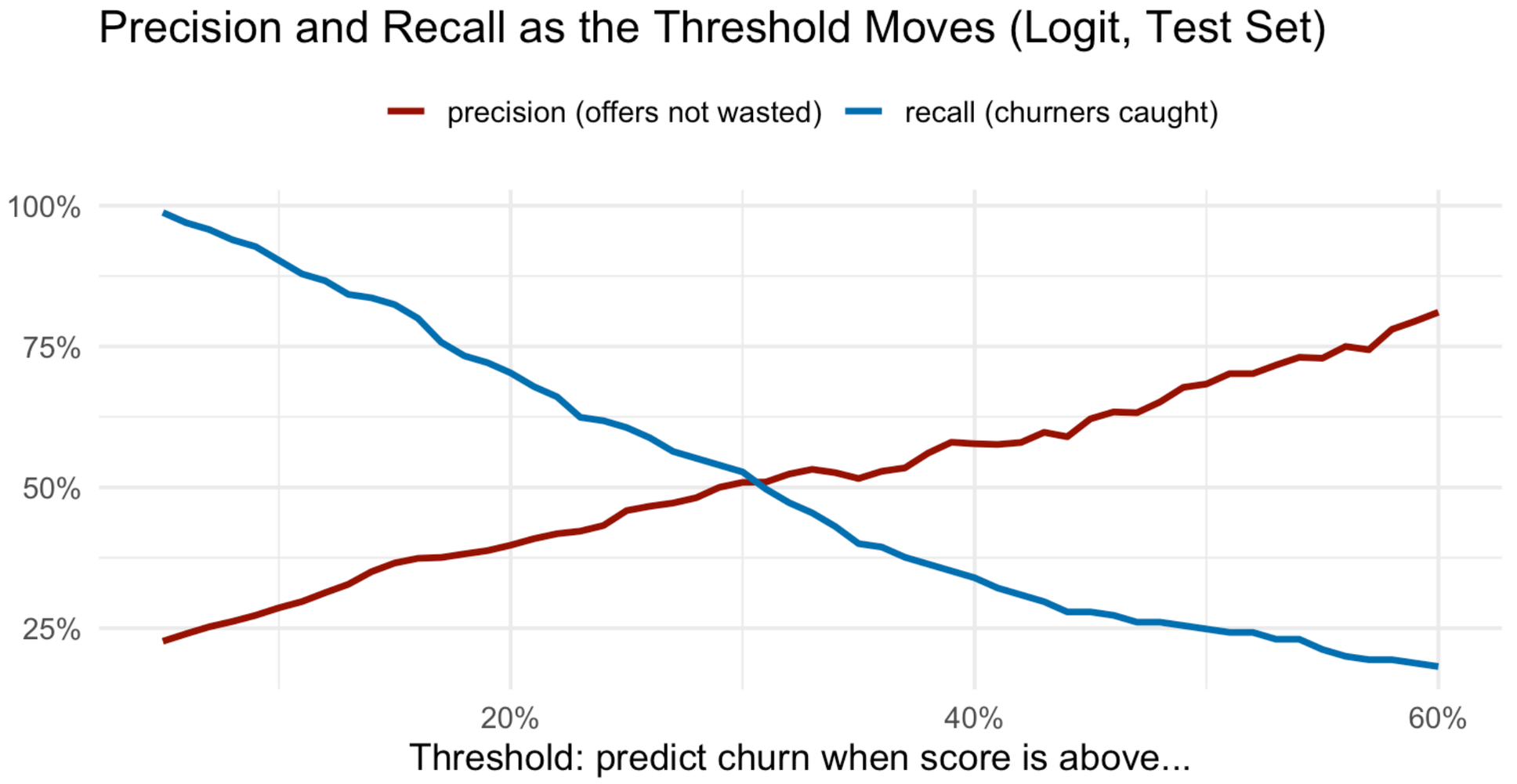

Each test customer gets a predicted churn probability. Grey: customers who stayed. Red: customers who actually churned. What do you see?

# Reading the scores



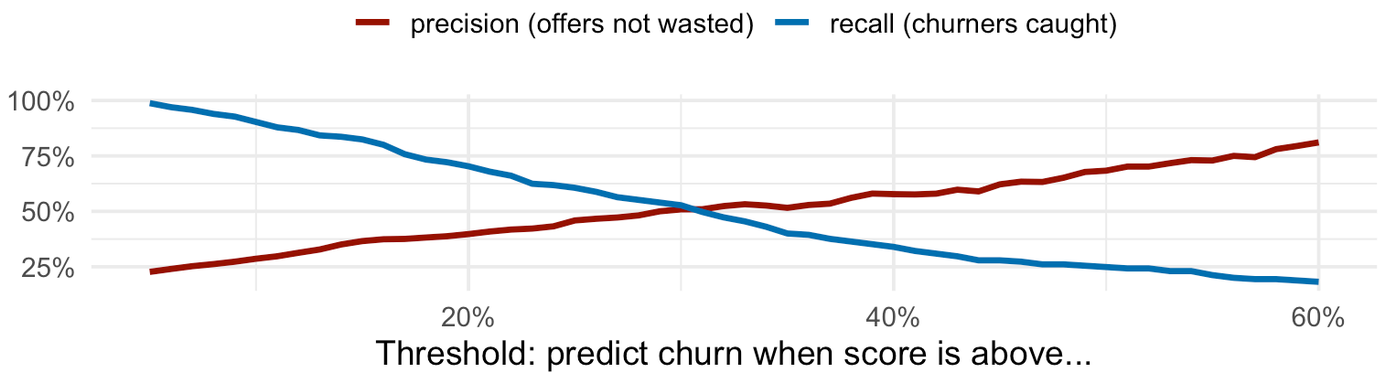

- The model **separates** the groups, imperfectly: churners (red) pile up at higher scores, stayers at lower ones, and the middle **overlaps**
- To act, we must draw a line: a **threshold**. Everyone to the right gets the retention offer, everyone to the left does not
- Wherever we draw it, we make both mistakes: some grey to the right (wasted offers), some red to the left (missed churners). The threshold only chooses the **mix**

# The threshold is a dial




The same logit, scored at every threshold from 5% to 60%. What happens to each line as the threshold rises, and where would you set the dial?

# Reading the dial




- **Raise the threshold** (only customers with very high scores get called churners): the few calls you still make are usually correct, so **precision rises**. But real churners with medium scores no longer make the cut, so you miss them: **recall falls**
- **Lower the threshold** (customers with even modest scores get called churners): you now catch nearly every real churner, so **recall rises**. But many of the people you call were going to stay anyway, so **precision falls**
- There is no "correct" threshold: it is a **business choice**, and it follows from **which mistake costs more**
  - A **wasted offer** costs more (say the offer is a big discount): protect **precision**, so raise the threshold
  - A **lost customer** costs more (say each account is worth thousands): protect **recall**, so lower it
- 0.5 is just a default: with rare events the useful thresholds sit **well below** it

# So how do you pick the threshold?

The dial chart is a **menu**, and the business picks from it. Say the retention team insists that **at least half the offers land on real churners**:

1. **Turn the goal into a metric target**: the offers go to the customers we call churners, so "the share of offers that land on real churners" is, word for word, **precision** (of our "churn" calls, how many were right). The target: precision at or above 50%, with as much recall as that allows
2. **Read it off the dial chart** (on the test set): the lowest threshold that clears 50% precision is **29%**
3. **Check what it costs**: at a threshold of 29%, precision is 50.0% and recall is 53.9%: about half the churners still slip through. If that is unacceptable, the goal (or the model) has to change

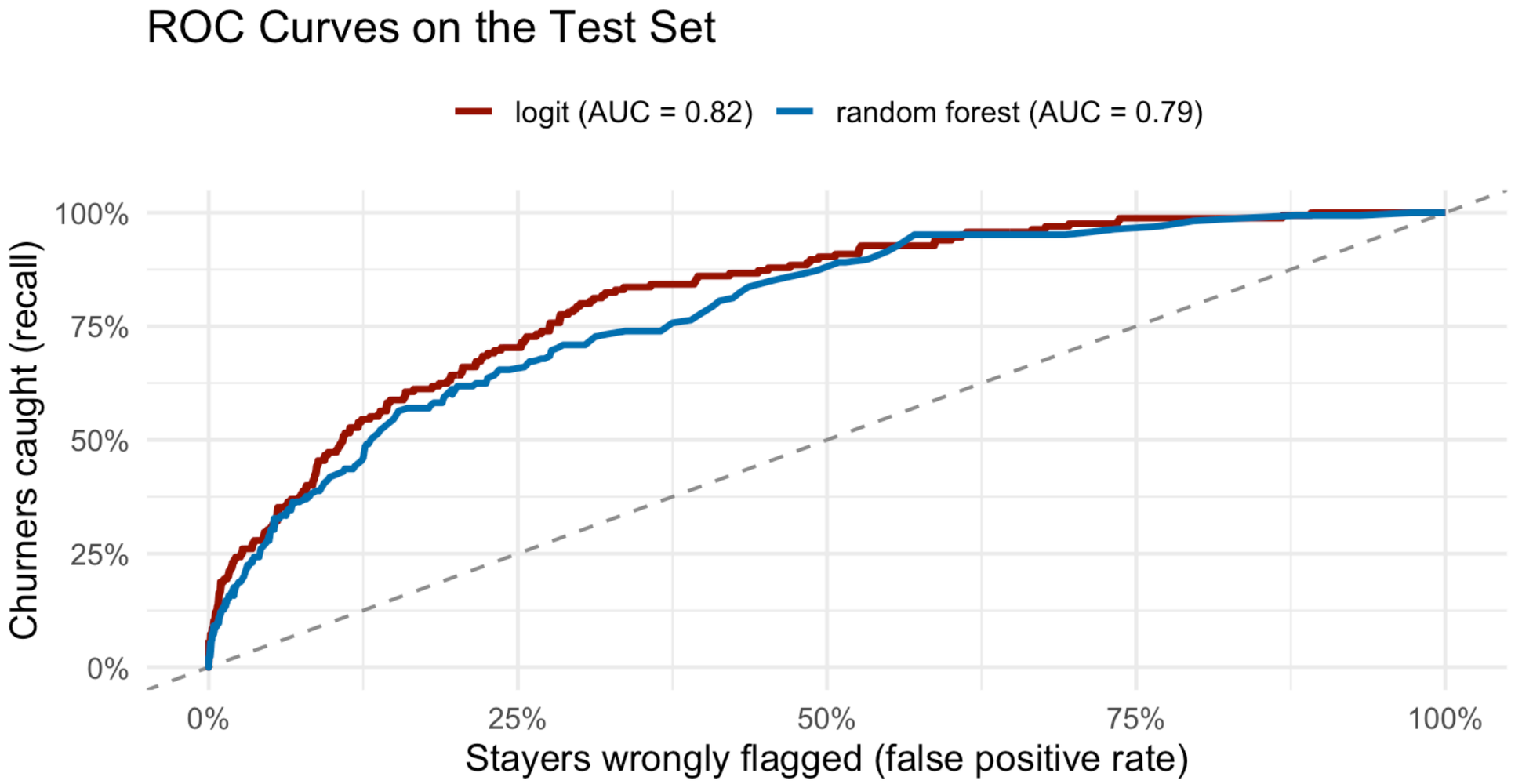
> Different goals pick different spots on the **same chart**: cheap offers favor recall (set the bar low), expensive offers favor precision (set it high). And when the team can only make so many calls (say, 90), skip the threshold entirely: rank the customers by score and work from the top of the list (coming up in a few slides).

# ROC and AUC: comparing models before choosing a threshold

The **ROC curve** traces the whole threshold dial at once: for every threshold, it plots the share of churners caught (y) against the share of stayers wrongly called churners (x).

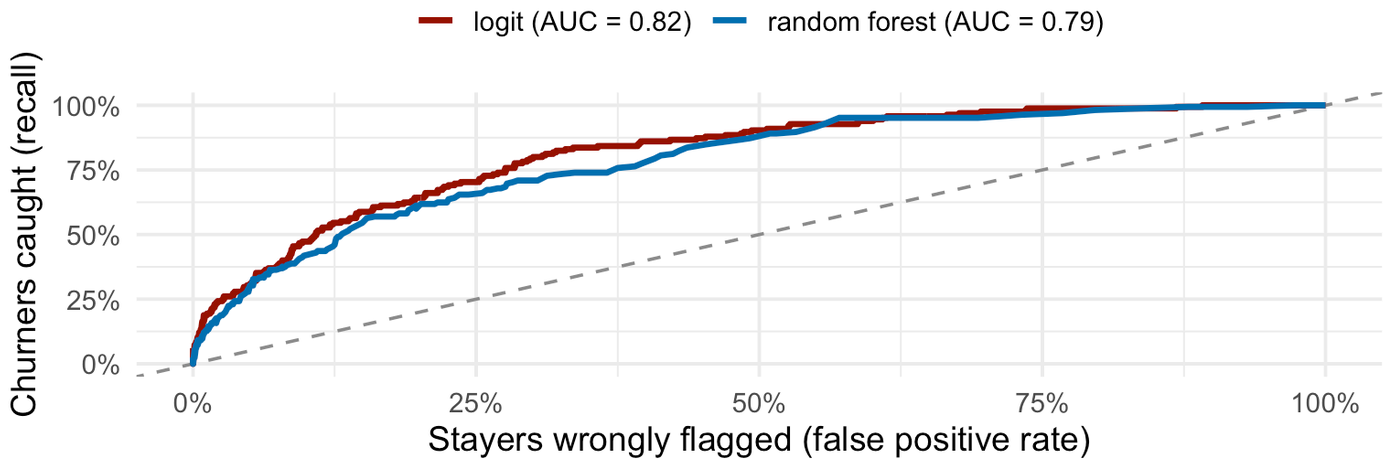
- **Top-left is best**: catch many churners, few false alarms
- The **diagonal** is coin-flipping
- **AUC** (area under the curve) summarizes it in one number: the chance the model scores a random churner **above** a random stayer. 0.5 = random, 1.0 = perfect
- AUC is **threshold-free**: good for comparing models early, before the business has picked its threshold

# ROC on our test set: logit vs. random forest




Both models, every threshold, one chart. The dashed diagonal is random guessing. Which model would you take to the retention team?

# Reading the ROC




- Both curves sit **well above the diagonal**: either model beats coin-flipping by a lot
- The baseline wins: AUC **0.82** (logit) vs. **0.79** (forest). The churn signal here is mostly **simple**: each feature pushes risk up or down on its own, which is exactly the kind of pattern a logit captures
- That is a common and useful result: the fancy model is not automatically better. When the logit matches or beats the forest, take the logit: **the same predictions, plus a story you can read**

# The recipe

What we just did, as steps you can rerun on any yes/no problem:

| Step | What to do |
|---|---|
| **1. Data** | one row per customer: the outcome (0/1) and the features |
| **2. Split** | 70/30 train/test, before touching any model |
| **3. Fit** | a logit first (the baseline), then a challenger (random forest) |
| **4. Score** | predicted probabilities on the **test set** only |
| **5. Compare** | AUC and the metrics table: does the challenger beat the baseline? |
| **6. Read** | odds ratios (logit), feature importance (forest): what drives risk |
| **7. Act** | pick the threshold that matches the business goal, or rank by score and act on the top of the list |

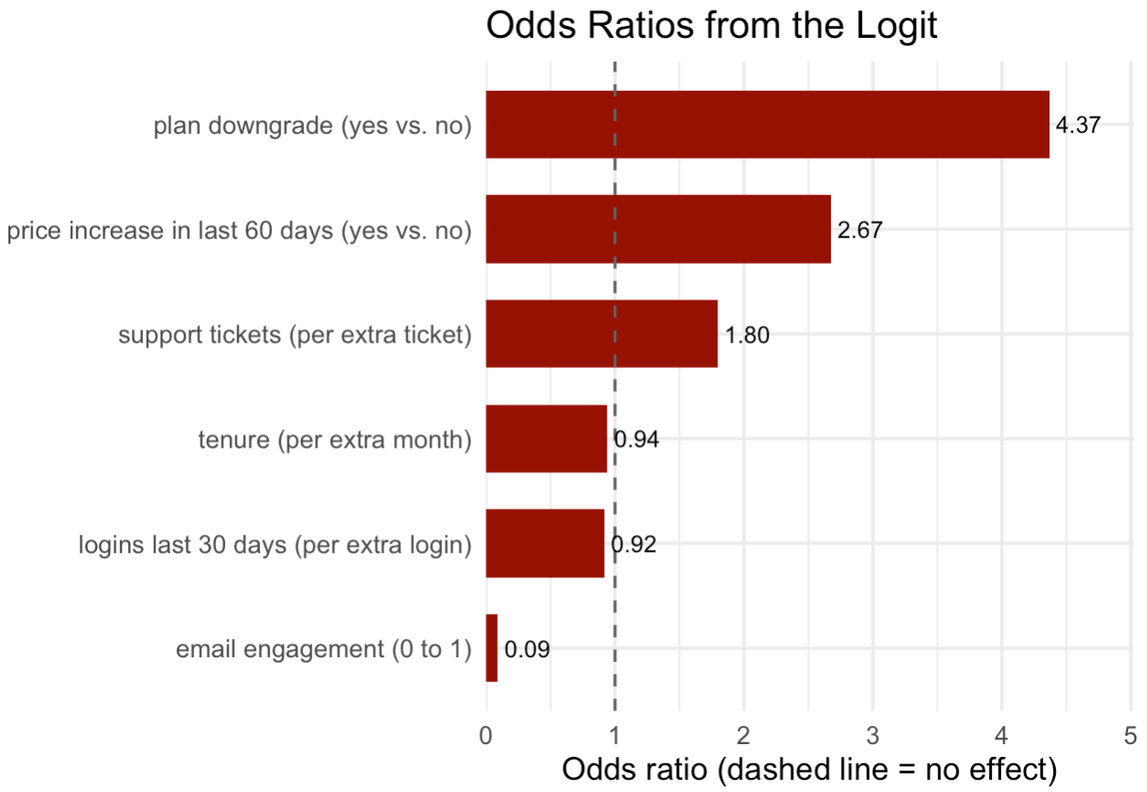
# Step 5: the metrics table

Both models on the test set, threshold at the 0.5 default:

| | Accuracy | Precision | Recall | AUC |
|---|---|---|---|---|
| **Logit** | 84.1% | 68.3% | 24.8% | 0.82 |
| **Random forest** | 83.1% | 65.1% | 17.0% | 0.79 |

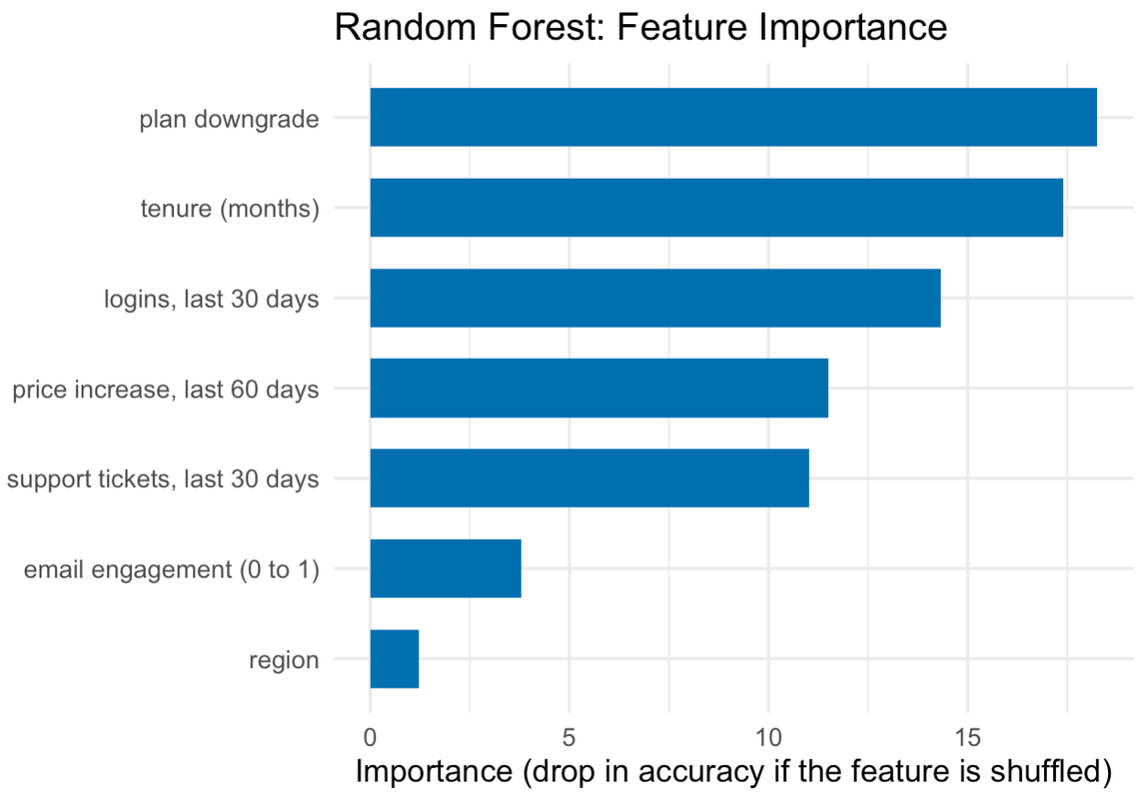
- Accuracy looks great for both, but churn is only 18.3% of customers: we know better than to stop there
- Read the logit row: when it says “churn” it is right 68.3% of the time (precision), but it catches only 24.8% of the real churners (recall). The reason: at the 0.5 cutoff it calls just **60 of the 901** test customers churners
- Why so few? Look back at the score histogram: hardly anyone scored above 50%, so a 0.5 cutoff barely ever fires. The fix is not a new model, it is a **lower threshold**: that is exactly how we picked 29% a few slides ago

# Step 6: what drives churn (logit odds ratios)




- Each bar turns a logit coefficient (week 5) into a **multiplier on the odds** of churn: a bar at 2.0 means that feature **doubles the odds**, holding the others fixed
- Right of the dashed line raises risk: a **plan downgrade** and a recent **price increase** are the loudest warnings, and every extra **support ticket** adds risk
- Left of the line protects: **engaged** customers and **long-tenure** customers churn less
- Week 5 logic, new job: the same odds ratios now tell the retention team **what to watch**

# Step 6: what the forest relied on




- The forest has no coefficients to read, so we test a feature by **breaking it**: mix up one column at random, so every customer gets **someone else's tenure** and the other columns stay put. Same numbers, **wrong customers**
- Predict again with the broken column: if the predictions get **much worse**, the forest was relying on it. Each bar is that **drop in accuracy**; near zero means the feature barely matters
- The ranking broadly **agrees with the logit**: the plan downgrade on top, then tenure, logins, and the price increase. Agreement across two different models is reassuring
- Importance says **what the model uses**, not why, and not causation: shuffling tells us about predictions, not about customers

# Step 7: act on the scores

Suppose the retention team can call **10% of the test customers** (91 calls). Skip the threshold: rank by score and take the **top 91** (the forest's list):

- **Precision at the top 10%**: 57.1% of the called customers were real churners (vs. a base rate of 18.3% if we called at random)
- **Recall at the top 10%**: the list catches 31.5% of all the churners in the test set
- That is the whole business case in two numbers: **how good the calls are**, and **how much of the problem they cover**

> Three ways to run the same model: the default 0.5 cutoff (fine when both mistakes cost about the same and the outcome is not rare), a threshold chosen to hit a precision or recall target, or **top-K** when the team can only handle so many customers. The model gives scores; **the business sets the dial.**

# What a classifier will not tell you

- **Prediction is not causation.** “Support tickets predict churn” does not mean tickets cause churn; unhappy customers file tickets *and* leave. The model finds **who** is at risk, not **what to change**
- **Garbage in, garbage out.** The model only sees the features you give it; a missing driver (a competitor’s promo) is invisible
- **The past must look like the future.** A model trained before a price change may break after it. Retrain, and re-test, on fresh data
- **A score is not a strategy.** Knowing who will churn is step one; the offer, and whether it works, is an experiment (coming later in the course)

# Practice: predicting ad clicks

Same recipe, new question: **who clicks the ad?**

- The exercise: fit a logit on 5,000 ad impressions, predict clicks at the 0.5 cutoff, build the **confusion matrix** by hand, then compute **accuracy, precision, recall, and F1**, with questions at every step
- Everything you need, also linked on the course page: the handout, the R Markdown template, and the data (put the template and the data in **one folder**, then knit)
- One thing to notice: clicks here are **31.8% of impressions**, not rare. The 0.5 default behaves reasonably on this data; compare that with our churn data (18.3%), where 0.5 barely ever fired
- Work through it before Thursday if you can: it is the best preparation for the homework

# Wrap-up

The R code and data behind every chart and table:

- `w9-code.zip`: the class script, the churn data, and the ad-click data for the exercise
- `w9-1-classifiers-class.R`: the churn model end to end: split, logit vs. forest, metrics, ROC, odds ratios, importance, top-K

Optional reading: chapters 11.4 to 11.6 of *R for Marketing Research and Analytics*. Thursday: **Hong Zhang (Netflix)**, bring questions about data science careers in marketing.

# Questions?